

Defect Reporting and Tracking

Agenda

- 1. What is Bug**
- 2. Defect Report**
- 3. Structure**
- 4. Life cycle of Bug**
- 5. Bug Tracking**
- 6. Conclusion**
- 7. Examples**

What is Bug

In testing process when a tester executes the test cases he might observe that the actual test results do not match from the expected results. The variation in the expected and actual results is known as defects, or bug.

Term "bug" used to describe inexplicable defects has been a part of engineering jargon for many decades and predates computers and computer software; it may have originally been used in hardware engineering to describe mechanical malfunctions.



Defect is difference between expected and actual result

Defect Report

A major test objective is to identify defects. Once identified, defects need to be recorded, monitored, reported and corrected.

After uncovering a defect (bug), testers generate a formal defect report.

Bug report is a technical document that describes failure mode in system under test .

Structure

Defect report or Bug report consists of the following information:

- **ID** – Unique identifier given to the defect.
- **Project** – Project name
- **Product** – Product name
- **Release Version** – Release version of the product
- **Detected Build Version** – Build version of the product where the defect was detected
- **Summary** – Brief summary of the defect
- **Description** – Detailed description of the defect.
- **Steps to Replicate** – Step by step description of the way to reproduce the defect.
- **Actual Result** – The actual result you received when you followed the steps.
- **Expected Results** – The expected results.
- **Attachments** – Attach any additional information like screenshots and logs.
- **Remarks** – Any additional comments to the defect.
- **Reported By** – The name of the person who reported the defect.



Structure

- **Assigned To** – The name of the person that is assigned to analyze/fix the defect.
- **Fixed by** – This field includes the details of the developer who fixed it like Name and ID
- **Date Closed** – This includes the Date when the bug is closed
- **Severity:** Based on the severity (Critical, Major or Minor) it tells us about impact of the defect or bug in the software application
- **Priority:** Based on the Priority set (High/Medium/Low) the order of fixing the defect can be made.



Structure

DEFECT SEVERITY CLASSIFICATION:

- **Critical** - this defect is causing system failure. Nothing can proceed further.
- **Major** - highly severe defect, is causing the system to collapse, however few parts of the system are still usable, and/or there are a few workarounds for using the system in the collapsed state too
- **Medium** - is causing some undesirable behavior, however system / feature is still usable to a high degree
- **Low** - is more of a cosmetic issue. No serious impedance to system functionality is noted.

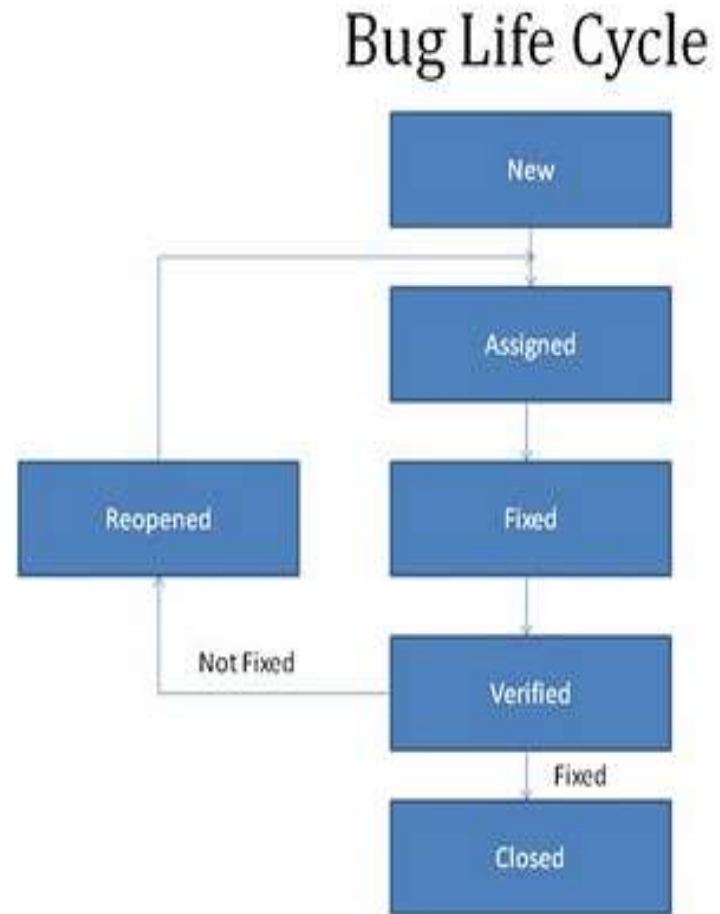
DEFECT PRIORITY CLASSIFICATION:

- **High**: Must be fixed in any of the upcoming builds but should be included in the release.
- **Medium**: May be fixed after the release / in the next release.
- **Low**: May or may not be fixed at all

Life cycle of bug

Defect life cycle is a cycle which a defect goes through during its lifetime. It starts when defect is found and ends when a defect is closed, after ensuring it's not reproduced. Defect life cycle is related to the bug found during testing.

The bug has different states in the Life Cycle. The Life cycle of the bug can be shown diagrammatically as follows:



Status



- **New** - When a defect is logged and posted for the first time
- **Assigned** - After the tester has posted the bug, the lead of the tester approves that the bug is genuine and he assigns the bug to corresponding developer and the developer team.
- **Open** - At this state the developer has started analyzing and working on the defect fix.
- **Fixed** - When developer makes necessary code changes and verifies the changes then he/she can make bug status as 'Fixed' and the bug is passed to testing team.
- **Verified** - The tester tests the bug again after it got fixed by the developer.
- **Reopened** - If the bug still exists even after the bug is fixed by the developer, the tester changes the status to "reopened". The bug goes through the life cycle once again.
- **Closed bug** - Once the bug is fixed, it is tested by the tester. If the tester feels that the bug no longer exists in the software, he changes the status of the bug to "closed". This state means that the bug is fixed, tested and approved.

Bug tracking

Bug tracking is a process used by quality assurance personnel and programmers to keep track of software problems and resolutions.

Bug tracking is important since complex software products can have tens of thousands of defects (or possibly more). Managing, evaluating and prioritizing these defects is a time consuming task. To help with this task, development teams use bug tracking software which are database systems that store defects and help team members manage issues.

Bug tracking systems include a database, which keeps track of facts pertaining to each bug. These facts might include the time a bug was reported, its severity, incorrect program behavior, details on how to recreate the bug, who reported the bug and what the programmers did to fix it. Bug tracking systems are associated with a bug's life cycle, which is tracked through the status assigned to each bug.



Conclusion

A software bug can cause faulty functionality, instability or even failure within an application or program. Fortunately, during the development phase, the software testing and quality assurance (QA) process addresses bug issues. The primary mission of testing teams and individual software testers is to comb through a program's code in order to detect mistakes and defects.

Defect reporting and tracking - it's very important and responsible part of testers work.

Quality indicators for good bug reports

- Clear for understanding(especially the summary)
- Actionable by development
- Move quickly from opened to closed
- Include only one problem per report



Example

Defect Report

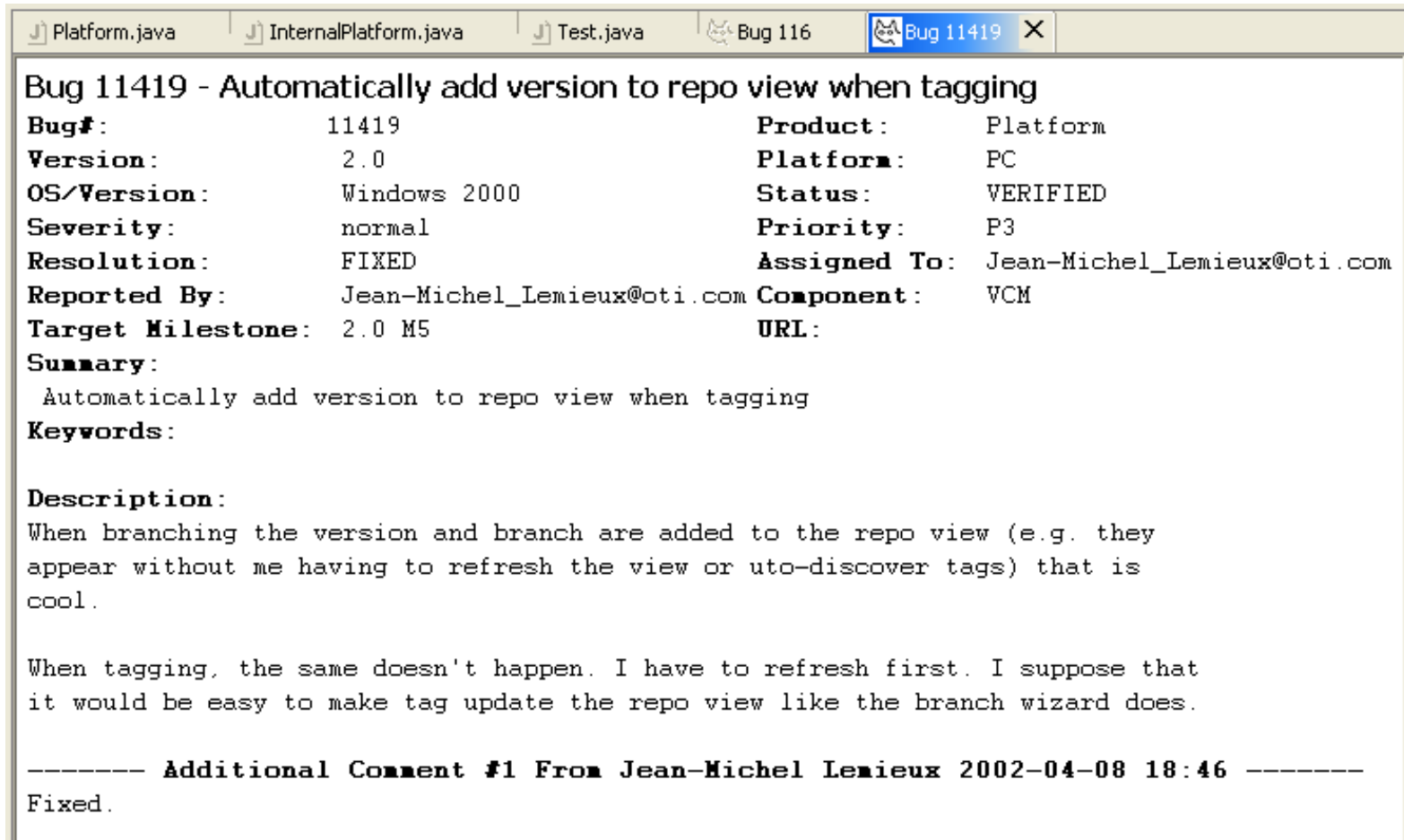
#	Summary	Steps to reproduce	Actual Result	Expected Result	Severity	Priority
1	New Comment Page – Active check box is checked in by default	1. Log in to app 2. Click on “New” link	“Active” checkbox is checked in by default	Active check box should be disabled by default.	High	High
2	New Comment Page- “Active” checkbox becomes active after page refresh	1. Log in to app 2. Click on “New” link 3. Check off “Active” check box 4. Click “Refresh” link	“Active” checkbox becomes checked in	“Active” checkbox should be checked off by default	Medium	Medium

Example

Defect Report

WEB APPLICATION TESTING - QA REPORT FOR floralandhardyofskippack.com									
ID	Test Cycle Name	Bug Description	Browser	Page Name/URL	Bug Type	Severity	Status	Date submitted	Valid and in scope (Customer's Approval)
11	WEB-QA01	For the Menu IN YOUR GARDEN sub menus are overlapped on main menu.	FIREFOX 3.6	http://www.floralandhardyofskippack.com/	GUI	MEDIUM	OPEN	14-10-2010	APPROVED
12	WEB-QA01	Enter the product "sedum" in search text box.The Search text link displays as "1 Sedum"	FIREFOX 3.6	http://www.floralandhardyofskippack.c	FUNCTIONAL	MEDIUM	OPEN	14-10-2010	APPROVED
13	WEB-QA01	Search functionality is not working for all the Items.	IE 7	http://www.floralandhardyofskippack.com/	FUNCTIONAL	HIGH	OPEN	14-10-2010	APPROVED
14	WEB-QA01	Price is not displayed for the items.	FIREFOX 3.6	http://www.floralandhardyofskippack.com/in-your-garden/perennials	TECHNICAL	SHOW STOPPER	OPEN	14-10-2010	APPROVED
QA Testing Summary									
Total Number of Defects			14						

Example



The screenshot shows a web browser window with several tabs: Platform.java, InternalPlatform.java, Test.java, Bug 116, and Bug 11419 (which is the active tab). The bug report content is as follows:

Bug 11419 - Automatically add version to repo view when tagging

Bug#:	11419	Product:	Platform
Version:	2.0	Platform:	PC
OS/Version:	Windows 2000	Status:	VERIFIED
Severity:	normal	Priority:	P3
Resolution:	FIXED	Assigned To:	Jean-Michel_Lemieux@oti.com
Reported By:	Jean-Michel_Lemieux@oti.com	Component:	VCM
Target Milestone:	2.0 M5	URL:	

Summary:
Automatically add version to repo view when tagging

Keywords:

Description:
When branching the version and branch are added to the repo view (e.g. they appear without me having to refresh the view or uto-discover tags) that is cool.

When tagging, the same doesn't happen. I have to refresh first. I suppose that it would be easy to make tag update the repo view like the branch wizard does.

----- **Additional Comment #1 From Jean-Michel Lemieux 2002-04-08 18:46** -----
Fixed.

Example

Bug Report #86

(Reported by whoister...@gmail.com, Oct 3, 2008)

Type: Defect, Status: Fixed, Closed: Oct 6, 2008)

Summary: NoSuchMethodError: No such method getHeight().

Description: ...

Expect decoded output, but see Error:

NoSuchMethodError: No such method getHeight().

...

Got the following trace:

com/google/zxing/common/BaseMonochromeBitmapSource.estimateBlackPoint. ALERT: java/lang/NoSuchMethodError: No such method getWidth()

Comment 1 by project member srowen, Oct 4, 2008

Exactly, sounds like the same thing. BaseMonochromeBitmapSource is an abstract class which implements an interface defining methods like getHeight() and getWidth(). The abstract class doesn't have to define an implementation for all the interface methods, and it doesn't define these ...But the Nokia JVMs don't seem to accept that...The workaround is that we can just add
public abstract int getHeight();
public abstract int getWidth();

...

Comment 4 by project member srowen, Oct 6, 2008

I will add those other two methods...

Thank you for attention



